Part II

Text-file processing

Text processing

- Linux is extremely good at simple text-file processing.
- This is useful for post-processing data-files, creating templated settings-files, etc.
- We have already seen `sort` and `uniq`.

Selecting lines with cat/head/tail/grep

- In a very long file, we might want to select certain lines only:
  - `cat Lecture_1.tex | head -n 10`

  selects the first 10 lines of `Lecture_1.tex`.
  - `tail -n 10 -f simulation.log`

  outputs the last 10 lines of `simulation.log`, and continues to run, outputing extra lines when the log-file is updated.
  - `grep -C 4 " head " Lecture_1_body.tex`

  selects all lines containing " head " as well as 4 lines before and after each occurrence (`-C` for context).
- With `-i` and `-A` you could find all lines containing `HEAD`/`HeAd`/`head`/etc. (`i`gnore-case) and 2 lines `A`fter each occurrence.

Selecting fields with cut

- The `cut` command allows us to select specific fields (or columns) from every line of a file.

```
$ ls -l | cut -f 3
total 4404
-rwx------ 1 pmb39 lsc     558 Sep 15  2023 analyze_servers.sh
-rw------- 1 pmb39 lsc      13 Sep 15  2023 analyze_servers.sh~
drwx------ 2 pmb39 lsc    4096 Oct 16  2023 auto
...
```

  ... doesn't seem to work.

- This is because `cut` assumes a TAB delimiter by default. To choose a different delimiter, use `-d`:

```
$ ls -l | cut -f 3 -d ' ' | head -10

pmb39
pmb39
pmb39
...
```

- We could apply `sort` and `uniq` to this to get a list of all file-owners in the current directory.

Simple character replacement with tr

- The `tr` command replaces one or more characters (tr-anslates) by another.
- For a simple Caesar Cipher (a→b, b→c, etc.):

```
echo "Beware the Ides of March" | tr 'a-z' 'b-za'
Bfxbsf uif Ieft pg Mbsdi
```

or, more correctly:

```
echo "Beware the Ides of March" | tr 'a-zA-Z' 'b-zaB-ZA'
Cfxbsf uif Jeft pg Nbsdi
```

- To **d**elete certain characters use `-d`.

```
echo "Words are harder to read without spaces" | tr -d ' '
Wordsareharderto readwithoutspaces
```

paste-ing

- The `paste` command takes sequential lines from multiple files and puts them on the same line.

file1.txt

```
Expressive
Tardy
Massive
```

file2.txt

```
Walrus
Petunia
Strawberry
```

- Combining these with `paste` gives:

```
$ paste file1.txt file2.txt
Expressive      Walrus
Tardy   Petunia
Massive Strawberry
```

joining

- `join` sounds similar, but requires an index field in both files to specify which lines are joined together.
- The files must be sorted according to the field used for joining.

join_1.txt

```
A Dog
B Cat
C Pig
E Lion
G Whale
I Eel
```

join_2.txt

```
B Cabbage
C Broccoli
D Sprout
E Lettuce
F Kale
G Bean
H Pea
```

```
$ join join_1.txt join_2.txt
B Cat Cabbage
C Pig Broccoli
E Lion Lettuce
G Whale Bean
```

- You can specify which field of each file to join on with `-j N` or `-1 N` and `-2 N`.

patch-ing

- We briefly covered `diff` to produce the difference between two files.
- The inverse command is `patch` which applies the difference file to update an existing file.

```
$ patch my_program.C fixes.diff
patching file my_program.C
```

- A more common usage is to take `.diff` files from `git` (see later) containing changes to multiple files and apply them all at once:

```
$ patch -p1 < changes.diff
```

Fundamental text-editing - sed

- Basic text-substitution on one or more text-files is sometimes necessary.
- If you have mis-capitalized an acronym, for example:

```
sed 's/Muscl/MUSCL/' Thesis.tex > Thesis_new.tex
```

- This will replace (almost) all occurences of "Muscl" with "MUSCL" throughout **Thesis.tex** and put the result into **Thesis_new.tex**
- This actually only replaces the first instance of "Muscl" on each line.
- To replace every occurence on each line:

```
sed 's/Muscl/MUSCL/g' Thesis.tex > Thesis_new.tex
```

where the extra 'g' stands for 'global'.

sed in-place

- If you are certain that your `sed` script is working properly, you can modify the files as they are processed:

```
sed -i 's/Muscl/MUSCL/g' Thesis.tex
```

- This means you can also do:

```
for f in Chapter*.tex; do
  sed -i 's/Muscl/MUSCL/g' $f;
done
```

  to replace all occurences throughout all chapters in your thesis.

- You will need to take care if you refer to "Muscles" anywhere in your thesis...

Escaping characters

- You may need to modify a script which contains paths:

```
sed 's/\/home\/pmb39\//\/home\/raid\/pmb39\//g'
    myScript.sh
```

  which replaces **/home/pmb39/** with **/home/raid/pmb39/**

- The / character needs to be escaped as otherwise it would be interpreted as the delimiter between separate parts of the replacement command.
- Other characters which need to be escaped are:

  `$ . [ ] ^ ? +`

- Another way of writing the above is:

```
sed 's%/home/pmb39/%/home/raid/pmb39/%g' myScript.sh
```

  where the % character is now the delimiter as it is the first character after the 's'.

Matching certain lines

- What if you only want to replace Muscl with MUSCL in a list:

```
1) Use the Muscl method (see Toro for details of Muscl)
2) Another line
3) A line with Muscl
Reference to Muscl.
```

- Here we can force **sed** only to make the replacement if the line starts with a number:

```
sed '/^[0-9]*)/s/Muscl/MUSCL/g' Thesis.tex
```

- The regular expression `^[0-9]*)` matches the list indicators.
- The result will be:

```
1) Use the MUSCL method (see Toro for details of MUSCL)
2) Another line
3) A line with MUSCL
Reference to Muscl.
```

More regular expressions

- Regular expressions in `sed` are used for pattern-matching text.
- Examples are:
  - `.` match any character
  - `*` match any number (including zero) of the preceding character
  - `[0-9]` match any digit
  - `^` match the beginning of a line
  - `$` match the end of a line
- These are *not* exactly the same as what your `bash` command-line will recognize.
- `sed` supports POSIX.2 Basic Regular Expressions (BRE)

Extended examples

- The regular expression: `wh*it*ch`
  will match all of
  `which witch whitch wich`
- The regular expression: `wh*it.`
  will match all of
  `white wits with` but not `which` or `wit`
  although '`wit ` ' would be matched as the '.' matches the space.
- To avoid matching 'wit ':
  `wh*it[^ ]`
  where `[^ ]` means to match everything except a ' '
- "Some people, when confronted with a problem, think 'I know, I'll use regular expressions.' Now they have two problems." - Jamie Zawinski

More sed

- More details of `sed` are available at `http://www.gnu.org/software/sed/manual/`
- Regular expressions are covered in detail at `http://www.zytrax.com/tech/web/regex.htm`
- Knowing about regular expressions can speed things up at the command-line as well as in text-replacement.
- Text-editors (such as `emacs`, `vi`, and even `VSCode`) can also deal with regular-expression search/replace but may have minor variations in syntax.

awk

- An easier to read and more powerful alternative to **sed** is **awk**.
- Unlike **sed** which deals with whole lines of characters, **awk** deals with data-fields.

```
cat data.txt | awk '{print $1 $3 $5;}'
```

would extract fields 1,3,5 from **data.txt**.
So can **cut**.

More awk

- However, `awk` can select lines corresponding to regular expressions:

```
cat data.txt | awk '$1 ~ /^[AB]/{print;}'
```

would print all lines where the first field starts with either “A” or “B”.

- Simple arithmetic can also be carried out:

```
ls -l | awk '{size = size+$5;} END{ print "Total = " size;}'
```

would print the total number of bytes taken by files and directories in the current one.

Summary

- Linux has a very wide range of text and data manipulation tools.
- Very complex operations can be carried out by combining them in the correct way.
- For more complex operations, a more general language such as Python may be better suited.